



30 DE NOVIEMBRE DEL 2023
PROYECTO FINAL

SEMINARIO DE SOLUCIÓN DE PROBLEMAS DE SISTEMAS BASADOS EN CONOCIMIENTO
DANIEL ALBERTO VAZQUEZ RAMIREZ
CUCEI - UNIVERSIDAD DE GUADALAJARA
Profesor Jorge de Jesús Gálvez Rodríguez

Proyecto Final

Introducción

La toma de decisiones es un proceso que llevamos a cabo todos los días siempre buscando la solución que mejor satisfaga nuestras necesidades, incluso muchas veces la toma de decisiones se convierte en un proceso tan sencillo e intuitivo que la necesidad de procesamiento para analizar la situación pasa a segundo plano. Cruzar la calle, esperar el transporte público a una hora específica, o cualquier situación cotidiana podemos realizarla de una manera rápida dado que contamos con experiencia y conocimiento que nos permite llegar a una solución óptima que mejor se adapte al contexto en el que nos encontramos. La toma de decisiones también se lleva a cabo por las organizaciones como implementar una estrategia de marketing, pero para esto si bien la experiencia del encargado del área es un factor que influye, el error humano se encuentra presente y buscamos una solución que mejor nos otorgue resultados positivos. Para esto y muchas situaciones más existen los algoritmos de Machine Learning, propuestos para llevar el contexto real hacia una solución computacional, el objetivo es que la computadora realice los cálculos pertinentes reduciendo el sesgo de error en la toma de decisiones. En el presente trabajo se plasman las evidencias del conocimiento y habilidades adquiridas en el curso con las soluciones a unos problemas otorgados, seleccionando el algoritmo que mejor se adapte para finalmente presentar las conclusiones correspondientes.

Desarrollo

Para el presente trabajo se hace la selección de 3 algoritmos diferentes que otorguen la solución a 3 problemas específicamente de clasificación. Para elegir los algoritmos inicialmente se tiene que analizar la información y entender que es lo que se busca solucionar. El primer problema que se tiene es un dataset de clasificación binaria titulado “Employee” el cual nos presenta diferentes parámetros para saber si un empleado se va o se queda (1 y 0). La información del dataset es la siguiente:

#	A	B	C	D	E	F	G	H	I
1	Education	JoiningYear	City	PaymentTier	Age	Gender	EverBenched	Experience	LeaveOrNot
2	Bachelors	2017	Bangalore	3	34	Male	No	0	0
3	Bachelors	2013	Pune	1	28	Female	No	3	1
4	Bachelors	2014	New Delhi	3	38	Female	No	2	0
5	Masters	2016	Bangalore	3	27	Male	No	5	1
6	Masters	2017	Pune	3	24	Male	Yes	2	1
7	Bachelors	2016	Bangalore	3	22	Male	No	0	0
8	Bachelors	2015	New Delhi	3	38	Male	No	0	0
9	Bachelors	2016	Bangalore	3	34	Female	No	2	1
10	Bachelors	2016	Pune	3	23	Male	No	1	0
11	Masters	2017	New Delhi	2	37	Male	No	2	0
12	Masters	2012	Bangalore	3	27	Male	No	5	1
13	Bachelors	2016	Pune	3	34	Male	No	3	0
14	Bachelors	2018	Pune	3	32	Male	Yes	5	1

En la tabla anterior se puede observar los valores independientes y el valor dependiente (Y) así como datos en formato de cadena, con

esto entendemos la complejidad y lo necesario a usar para poder trabajar sobre él.

El segundo problema que se tiene es el dataset de “Heart” el cual presenta las diferentes variables que influyen para que una persona tenga problemas en el corazón como se muestra a continuación:

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	age	sex	cp	trtbps	chol	fbs	restecg	thalachh	exng	oldpeak	slp	caa	thall	output
2	63	1	3	145	233	1	0	150	0	2.3	0	0	0	1
3	37	1	2	130	250	0	1	187	0	3.5	0	0	0	2
4	41	0	1	130	204	0	0	172	0	1.4	2	0	0	2
5	56	1	1	120	236	0	1	178	0	0.8	2	0	0	2
6	37	0	0	120	354	0	1	163	1	0.6	2	0	0	2
7	57	1	0	140	192	0	1	148	0	0.4	1	0	0	1
8	56	0	1	140	294	0	0	153	0	1.3	1	0	0	2
9	44	1	1	120	263	0	1	173	0	0	2	0	0	3
10	52	1	2	172	199	1	1	162	0	0.5	2	0	0	3
11	57	1	2	150	168	0	1	174	0	1.6	2	0	0	2
12	54	1	0	140	239	0	1	160	0	1.2	2	0	0	2
13	48	0	2	130	275	0	1	139	0	0.2	2	0	0	2
14	49	1	1	130	286	0	1	171	0	0.6	2	0	0	2
15	64	1	3	110	211	0	0	144	1	1.8	1	0	0	2
16	58	0	3	150	283	1	0	162	0	1	2	0	0	2
17	50	0	2	120	219	0	1	158	0	1.6	1	0	0	2

Tenemos mayor cantidad de variables independientes, y en este caso solo se tienen valores numéricos. Este es un problema de clasificación binaria.

El tercer y último problema es titulado “Zoo”, el cual se presentan las características de un animal y su clasificación dentro del reino animal:

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	animal_name	hair	feathers	eggs	milk	airborne	aquatic	predator	toothed	backbone	breathes	venomous	fins	legs	tail	domestic	carnivore	class_type
2	antelope	1	0	0	1	0	0	1	1	1	1	0	0	4	0	0	1	1
3	antelope	1	0	0	1	0	0	0	1	1	1	0	0	4	1	0	0	1
4	bass	0	0	1	0	0	1	1	1	1	0	0	1	0	0	0	0	4
5	bear	1	0	0	1	0	0	1	1	1	0	0	0	4	0	0	0	1
6	beaver	1	0	0	1	0	0	1	1	1	0	0	0	4	1	0	0	1
7	buffalo	1	0	0	1	0	0	0	1	1	1	0	0	4	1	0	0	1
8	cat	1	0	0	1	0	0	0	1	1	1	0	0	4	1	1	1	1
9	cow	0	0	1	0	0	1	0	1	1	0	0	1	0	1	1	0	4
10	crabfish	0	0	1	0	0	1	1	1	1	0	0	1	0	0	0	0	4
11	cow	1	0	0	1	0	0	0	1	1	1	0	0	4	0	1	0	1
12	chickadee	1	0	0	1	0	0	1	1	1	1	0	0	4	1	0	1	1
13	chickadee	0	1	1	0	1	0	0	0	1	1	0	0	2	1	1	0	2
14	chub	0	0	1	0	0	1	1	1	1	0	0	1	0	1	0	0	4
15	clown	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	7
16	crab	0	0	1	0	0	1	1	0	0	0	0	0	4	0	0	0	7
17	crayfish	0	0	1	0	1	1	0	0	0	0	0	0	4	0	0	0	7
18	cow	0	1	1	0	1	0	1	0	1	1	0	0	2	1	0	0	2
19	deer	1	0	0	1	0	0	0	1	1	1	0	0	4	1	0	1	1
20	dogfish	0	0	1	0	0	1	1	1	1	0	0	1	0	1	0	0	4
21	dogfish	0	0	0	1	0	1	1	1	1	1	0	1	0	1	0	1	1
22	elbow	0	1	1	0	1	0	0	0	1	1	0	0	2	1	1	0	2

#	A	B	C	D	E	F	G	H	I
1	Class_Num	Number_Of	Class_Type	Animal_Names					
2	1	41	Mammal	aardvark, antelope, bear, boar, buffalo, calf, cavy, cheetah, deer, dolphin, elephant, giraffe, goat, gnu, hare, hippo, human, kangaroo, koala, lion, moose, monkey, mouse, pig, rabbit, ram, rhinoceros, seal, sheep, squirrel, tiger, walrus, whale, wolf, zebra					
3	2	20	Bird	chicken, crow, dove, duck, flamingo, gull, hawk, kiwi, lark, ostrich, parakeet, pigeon, quail, raven, robin, sparrow, starling, swan, turkey, vulture, wren, woodpecker, zebra					
4	3	5	Reptile	pitviper, seahorse, slowworm, tortoise, tuatara					
5	4	13	Fish	bass, carp, catfish, chub, dogfish, haddock, herring, pike, piranha, seahorse, shark, swordfish, trout					
6	5	4	Amphibian	frog, frog, newt, toad					
7	6	8	Bug	beetle, butterfly, caterpillar, cicada, cricket, grasshopper, ladybug, mantis, roach, scorpion, spider, tick, wasp					
8	7	10	Invertebrate	clam, crab, crayfish, lobster, octopus, scorpion, seahorse, slug, starfish, worm					

Este problema cuenta con dos datasets, uno relacionado a la clasificación y el segundo es informativo sobre las etiquetas. Después de analizarlo se observa que la columna de nombre es irrelevante por lo que se tendrá que eliminar cuando se trabaje con los datos, al igual se trata de un problema de clasificación pero en este caso no es binaria. Después de analizar e identificar las principales características de los problemas concluyo el utilizar el algoritmo de regresión logística binaria, knn y red neuronal multicapa, además que por su sencillez de implementación es una buena propuesta de solución.

Para trabajar con los datos de las columnas de tipo de cadena se usará label encoding para el trabajo con valores numéricos, se busca el estandarizar dado que existen columnas con un rango bastante amplio el cual nos puede producir error sensibles en la respuesta final. También se divide el dataset en train y test. Para el ajuste de pesos se usa el descenso de gradiente.

Como se ha venido trabajando a lo largo del curso no se usan modelos hechos con librerías, todo se realiza con funciones hechas por el estudiante. La implementación de regresión logística para los problemas se implementa de la siguiente manera, recalando que para el problema de zoo se usa una variante con la función softmax.

```
#Carga del dataset
data = pd.read_csv('Employee.csv')

headers = data.columns
df = data.to_numpy()

#Iniciliar pesos de forma aleatoria (8 + 10)
w = np.random.uniform(0, 10, size=9)
alpha = 5.8
#print("Pesos Iniciales: ",w)
#print("Pesos Iniciales: \n",w)
#-----#

#Vectorizacion
Educator(0), Joining Year(1), City(2), Gender (5), EverBenched(6)
label_encoder = LabelEncoder()
df[:, 0] = label_encoder.fit_transform(df[:, 0])
df[:, 1] = label_encoder.fit_transform(df[:, 1])
df[:, 2] = label_encoder.fit_transform(df[:, 2])
df[:, 4] = label_encoder.fit_transform(df[:, 4])
df[:, 5] = label_encoder.fit_transform(df[:, 5])
df[:, 6] = label_encoder.fit_transform(df[:, 6])

#Definir X
x = df[:, :-1]
#Definir Y
y = df[:, -1]
```

```
def calc(a,b):
    z = 0
    for j in range(len(a)):
        z = z + a[j] * b[j+1]
    return z + b[0]

def sigmoide(c):
    return 1 / (1 + np.exp(-c))

def cross_entropy(y1,y2):
    m = len(y1)
    epsilon = 1e-15
    b = 0
    for i in range(m):
        b = b + (y1[i] * np.log(y2[i] + epsilon) + (1 - y1[i]) * np.log(1 - y2[i] + epsilon))
    z = (-1/m) * b
    return z

def train():
    for i in range(0, len(X_train)):
        yp_train[i] = sigmoide(calc(X_train[i, :], w))
        #print(yp_train[i])
```

```
#Actualizacion de pesos
def actualizar(pesos):
    for j in range(0, len(pesos)):
        w[j] = w[j] - (alpha * pesos[j])

yp_test = np.zeros(len(y_test))
def test():
    #print(w)
    for i in range(0, len(X_test)):
        yp_test[i] = sigmoide(calc(X_test[i, :], w))
        #print(yp_test[i], y_test[i])
```

La función de error para evaluar los problemas es la entropía cruzada binaria. La diferencia entre el algoritmo implementado para Heart y Employee radica en el learning rate y en la cantidad de variables independientes. Para la aplicación de regresión logística como se mencionó anteriormente se usa la función softmax o la regresión logística multinomial. Específicamente se implementa una herramienta llamada one hot coding.

```
def numerador(pesos, variables):
    z = 0
    for j in range(len(variables)):
        z = z + (variables[j] * pesos[j])
    return np.exp(z + pesos[0])

def denominador(pesos, variables):
    z = 0
    for k in range(0, 7):
        s = 0
        for h in range(0, len(variables)):
            s = s + (pesos[h, k] * variables[h])
        z = z + np.exp(s + pesos[k, 0])
    return z

def funError(pred, train):
    epsilon = 1e-15
    er = 0
    pred = np.clip(pred, epsilon, 1 - epsilon)
    for k in range(0, 7):
        er = er + (train[k] * (np.log(pred[k])))
    return -er

def train():
    for i in range(len(X_train)):
        for j in range(0, 7):
            n = numerador(w[j, :], X_train[i, :])
            d = denominador(w, X_train[i, :])
            #softmax
            yp_train[i, j] = (n/d)
```

Específicamente se calcula numerador y el denominador como se presenta en la siguiente fórmula:

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$

Entropía cruzada binaria

$$CE = -\frac{1}{n} \sum_{j=1}^n \sum_{i=1}^c y_i \log \hat{y}_i$$

Para la función softmax se usa una variante de la entropía cruzada denominada como “categórica”.

KNN

Para la aplicación del algoritmo de Knn se busca categorizar datos nuevos coincidentes con el aprendizaje obtenido buscando otorgarle una variable en y. Como se sabe este algoritmo calcula la distancia con los demás puntos encontrados en el dataset y se categoriza con el valor en la etiqueta mayor repetido el número de vecinos elegidos.

```
#print(len(X_train))
k = 3

def etiquetar(y1, pos):
    orden = np.argsort(y1[:, 0])
    y1ord = y1[orden]
    v1 = 0
    v0 = 0
    for q in range(k):
        if(y1ord[k, 1] == 1):
            v1 = v1 + 1
        if(y1ord[k, 1] == 0):
            v0 = v0 + 1

    if(v1 > v0):
        yp_test[pos] = 1
    if(v0 > v1):
        yp_test[pos] = 0

def eucl(x1, x2, pos):
    d = np.zeros((len(x2), 2))
    for j in range(len(x2)):
        r = 0
        for k in range(len(x1)):
            r = r + ((x1[k] - x2[j, k])**2)
        d[j, 0] = np.sqrt(r)
        d[j, 1] = y_train[j]

    #print(d)
    etiquetar(d, pos)
```

La base del algoritmo se centra en varios puntos importantes, inicialmente es calcular la distancia del nuevo dato con cada uno en el dataset, claramente se guardan estos valores numéricos en una matriz junto con su etiqueta. Una vez obtenida la matriz de distancias y etiquetas, se ordena de menor a mayor por la columna de distancias, y se usa el número de k elegido para hacer la comparación correspondiente con el valor de la etiqueta mayor repetido y se etiqueta el registro. Las etiquetas se guardan en un vector el cual se compara con los valores reales de la variable dependiente. A diferencia del algoritmo anterior no se divide el dataset en conjuntos entrenamiento y prueba.

Red Neuronal Multicapa

Por último se implementa una red neuronal para buscar la solución a los mismos problemas, la ventaja encontrada es que internamente es una suma ponderada que se repite n veces en cada capa usando una función de activación y usando el error para la actualización de pesos. En este algoritmo se utiliza función sigmoide para obtener los valores entre 0 y 1

$$f(x) = \frac{1}{1 + e^{-x}}$$

Definida en la siguiente clase:

```
#funcion de activacion sigmoide
def funcionAct(n):
    #return 1/(1 + math.e **(-n))
    return round(1/(1 + math.e **(-n)))
```

La función para evaluar el costo es el error cuadrático medio. En este caso el dataset se divide en train y test. Es importante recalcar que en este caso se definen distintas matrices de pesos para cada capa, estas son algunas de las funciones creadas para el software.

```
#print(w1)
def sigmoide(c):
    return 1 / (1 + np.exp(-c))

def calculo(x1, w):
    n = np.zeros(ncapas1)
    for j in range(ncapas1):
        for k in range(len(x1)):
            n[j] = n[j] + x1[k] * w[j, k + 1]
        n[j] = n[j] + w[j, 0]
    return n

def salida(c):
    for i in range(len(c)):
        c[i] = sigmoide(c[i])
    return c

def salir(x1, w):
    n = 0
    for i in range(len(x1)):
        n = n + (x1[i] * w[i + 1])
    n = n + w[0]
    return n
```

La actualización de pesos es un tanto complejo entender en cuanto a la hora de implementar dado la variedad de derivadas encontradas, lo importante y la base es entender que la actualización se basa en los valores obtenidos en el momento de obtener la salida final, obtener el costo y retro propagar hacia atrás el error para la actualización.

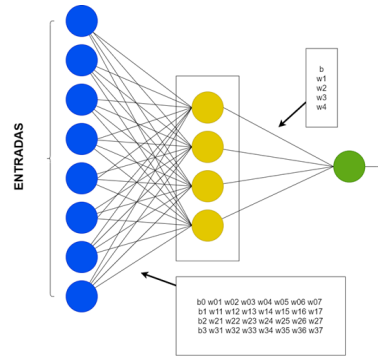
```
def actualizacion1(capa, y, yreal, x):
    for i in range(len(capa)):
        d = (y - yreal) #derivadas error cuadrático medio
        #d = - ((yreal/y) + ((1 - yreal)/(1 - y))) #entrop
        d = d * (y * (1 - y)) #sign
        #error de la última capa
        a = actualizacion2(d, w3[i+1], capa[i])
        for j in range(0):
            w1[i, j + 1] = w1[i, j + 1] - (alpha * (a * x[j]))
            w1[i, 0] = w1[i, 0] - (alpha * a)
        d = d * capa[i] #salida de capa
        w3[i+1] = w3[i+1] - (alpha * d) #actualizacion
        w3[0] = w3[0] - (alpha * ((y - yreal)*(y * (1 - y))))
    #return aux

def actualizacion2(e, wa, capa):
    res = (capa * (1 - capa))
    res = res * wa
    return res

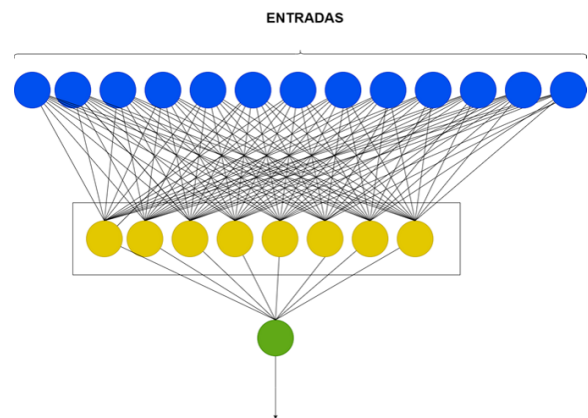
def train():
    #error = 0
    for i in range(len(X_train)): #len(X_train)
        capal = calculo(X_train[i, :], w1)
        capals = salida(capal)
        y = salir(capals, w3)
        yp_train[i] = sigmoide(y)
        #print(yp_train[i], y_train[i])
        actualizacion1(capals, yp_train[i], y_train[i], X_train[i, :])
    #return #error = error de train[i] en train[i]
```

Los modelos usados para la solución de employee y heart es el siguiente:

Employee



Heart



Resultados

Para evaluar el rendimiento de los algoritmos implementados se usarán las siguientes métricas: accuracy, precision, sensitivity, f1 score y una matriz sencilla de confusión.

Regresión logística

Employee:

```
Accuracy: 71.21374865735768
Precision: 63.730569948186535
Sensitivity: 38.31775700934579
F1 Score: 47.859922178988334
TP: 123
FP: 70
TN: 540
FN: 198
```

	Real	
Prediction	Positive (1)	Negative (0)
	TP: 123	FP: 70
	F.N: 198	TN: 540

Heart

```
Accuracy: 78.68852459016394
Precision: 78.78787878787878
Sensitivity: 81.25
F1 Score: 80.0
TP: 26
FP: 7
TN: 22
FN: 6
```

	Real		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 26	FP: 7
	Negative (0)	F.N: 6	TN: 22

```
Accuracy: 74.72527472527473
Precision: 75.47169811320755
Sensitivity: 80.0
F1 Score: 77.66990291262137
TP: 40
FP: 13
TN: 28
FN: 10
```

	Real		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 40	FP: 13
	Negative (0)	F.N: 28	TN: 10

Zoo

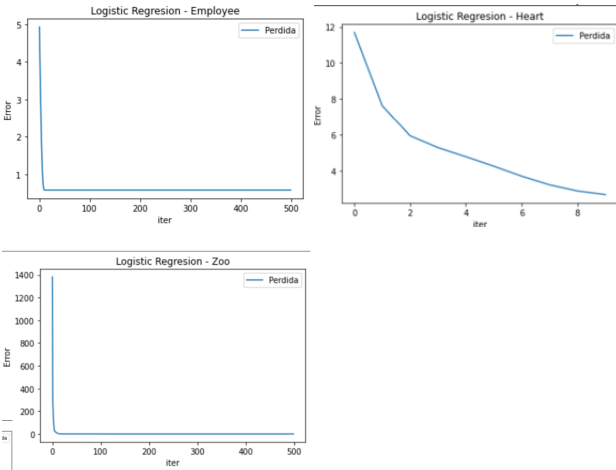
```
Accuracy: 95.23809523809523
Precision: 95.23809523809523
Sensitivity: 95.23809523809523
F1 Score: 95.23809523809523
TP: 19
FP: 1
TN: 105
FN: 1
```

	Real		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 19	FP: 1
	Negative (0)	F.N: 1	TN: 105

Zoo:

```
Accuracy: 90.47619047619048
Categoria: 1
Precision: 91.66666666666666
Sensitivity: 100.0
F1 Score Cateogira: 95.65217391304347
Categoria: 2
Precision: 100.0
Sensitivity: 100.0
F1 Score Cateogira: 100.0
Categoria: 3
Precision: 0.0
Sensitivity: 0.0
F1 Score Cateogira: 0.0
Categoria: 4
Precision: 100.0
Sensitivity: 50.0
F1 Score Cateogira: 66.66666666666667
Categoria: 5
Precision: 0.0
Sensitivity: 0.0
F1 Score Cateogira: 0.0
Categoria: 6
Precision: 100.0
Sensitivity: 100.0
F1 Score Cateogira: 100.0
Categoria: 7
Precision: 100.0
Sensitivity: 100.0
F1 Score Cateogira: 100.0
```

Error:



	Real Clase 1		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 11	FP: 0
	Negative (0)	F.N: 1	TN: 9

Red Neuronal

Employee

```
CUCEI/Sem Sistemas Basados/Proyecto')
Accuracy: 64.55424274973147
Precision: 45.714285714285715
Sensitivity: 14.953271028037381
F1 Score: 22.535211267605632
TP: 48
FP: 57
TN: 553
FN: 273
```

	Real		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 48	FP: 273
	Negative (0)	F.N: 57	TN: 553

KNN

Employee

```
Accuracy: 74.11385606874329
Precision: 63.0718954248366
Sensitivity: 60.12461059190031
F1 Score: 61.56299840510366
TP: 193
FP: 113
TN: 497
FN: 128
```

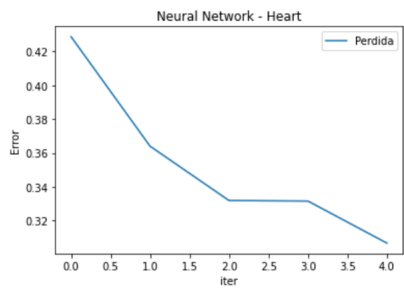
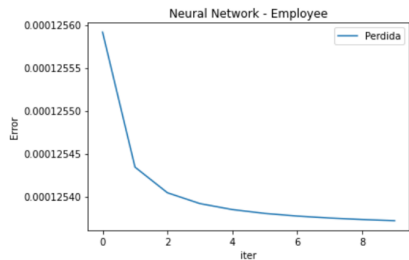
	Real		
Prediction		Positive (1)	Negative (0)
	Positive (1)	TP: 193	FP: 113
	Negative (0)	F.N: 128	TN: 497

Heart

Heart

```
Accuracy: 75.40983606557377
Precision: 77.41935483870968
Sensitivity: 75.0
F1 Score: 76.19047619047619
TP: 24
FP: 7
TN: 22
FN: 8
```

Real			
Prediction	Positive (1)		Negative (0)
	Positive (1)	TP: 24	FP: 7
	Negative (0)	F.N: 8	TN: 22



Conclusión

Finalmente para concluir con el presente trabajo de proyecto final sobre modelos de clasificación y machine learning puedo llegar a rescatar los siguientes puntos importantes después del análisis e implementación de los modelos. La regresión logística binaria fue el mejor algoritmo en el problema de Zoo. En el caso particular del problema de heart en regresión logística se identificó que un incremento en las iteraciones genera un overfitting considerable, por lo que se recomienda tener cuidado y revisar estos casos en las pruebas. En cuanto a los problemas que contienen diferentes categorías la regresión logística binaria no es un buen modelo a utilizar, en su caso es mejor utilizar la función softmax o multinomial. Para implementar cualquier modelo siempre será importante el análisis de los datos y aplicar las técnicas correctas que nos permitan trabajar con los datos como el label encoding o standar scaler. En relación al algoritmo de KNN es de suma importancia hacer pruebas con diferentes números de vecinos, ya que con lo realizado se observó que entre más vecinos la predicción es incorrecta. Por último punto a revisar es la red neuronal que es un

modelo bastante interesante pero también es complejo y un poco delicado dado que influyen bastantes factores desde el número de capas, neuronas, función de activación, función de error, learning rate y la correcta actualización de pesos. El algoritmo con mejor desempeño en la clasificación fue el Knn.

Referencias

1. “Qué es Retropropagación Concepto y definición. Glosario”. GAMCO, SL. Accedido el 1 de diciembre de 2023. [En línea]. Disponible: <https://gamco.es/glosario/retropropagacion/>
2. “Regresión Softmax | Interactive Chaos”. Home page | Interactive Chaos. Accedido el 1 de diciembre de 2023. [En línea]. Disponible: <https://interactivechaos.com/es/manual/tutorial-de-machine-learning/regresion-softmax>
3. Métricas de Clasificación. <https://www.themachinelearners.com/metricas-de-clasificacion/#:~:text=La%20m%C3%A9trica%20accuracy%20representa%20el,clasificados%2C%20tanto%20positivos%20como%20negativos.&text=Es%20recomendable%20utilizar%20esta%20m%C3%A9trica,n%C3%BAmero%20de%201s%20y%200s>
4. “Descubra el algoritmo KNN : un algoritmo de aprendizaje supervisado”. Formation Data Science | DataScientest.com. Accedido el 1 de diciembre de 2023. [En línea]. Disponible: <https://datascientest.com/es/que-es-el-algoritmo-knn>